

How We Evaluate

You get the categories and the weights. We keep the detailed scoring bands internal, so that you build the best system you can rather than the system that games a checklist.

Two gates come before any of this. If the stack does not come up with one command, or there is no reachable public URL and no demo video, we cannot review the submission at all. See `03-deliverables-and-submission.md`.

Weights

Weight	Category	What we are looking at
25%	Fault localization	Does it actually find the fault, and is the reasoning sound? Correct handling of the live/dark boundary. Grouping many dark poles into one incident. Multiple simultaneous faults. A defensible answer for the 60% of transformers with no recorded pole ordering. Robustness to missing, duplicate, late, and out-of-order telemetry. Honest confidence reporting.
20%	Product judgment	Did you solve the department's problem or the problem that was easiest to build? What you chose to include and exclude. Whether false positives were taken seriously. Whether the AI feature you added is in a place where it earns its keep — and whether you can argue for it.
20%	Architecture and data design	Whether your ingestion design survives contact with 39 msg/s and a 5,000-message burst. Your topology representation. Schema quality. API design. Whether the design would extend from one subdivision to thirty without a rewrite, and whether you know where it wouldn't.
15%	Operator experience	Is this usable by a non-engineer at 2 a.m.? Information hierarchy — does the most important thing dominate the screen? Map and list working together. How ambiguity and low confidence are communicated. Whether the ticket workflow matches how the work actually happens.
15%	Documentation and reproducibility	Whether we could run, understand, and hand off your system without talking to you. Architecture doc matching reality. The deployment troubleshooting section. Quality of your decision log and assumptions.
5%	Engineering craft and AI leverage	Tests on the logic that matters. Commit history that shows how you worked. Your AI workflow write-up — specifically, evidence that you can distinguish good AI output from bad.

What moves the needle most

Since you have limited hours, here is where they are best spent:

Get the localization right and explain it well. This is a quarter of the score and it is also what most submissions get wrong. A system that finds the correct span and explains its reasoning clearly, with a plain UI, scores far better than a beautiful dashboard that alerts on every dark pole.

Treat the missing topology as the main problem, not an edge case. It affects the majority of the network. A submission that quietly assumes complete wiring data has skipped the assignment's central difficulty.

Make it run. Reproducibility and documentation together are 15%, and they also gate everything else. Time spent making `docker compose up` bulletproof is never wasted.

Say what is broken. Every submission has rough edges. Candidates who document theirs consistently score higher than those who hope we won't notice, because we do notice, and the difference between "known and documented" and "apparently unaware" is large.

What actively costs you

- One alert per dark pole instead of one per fault.
- No distinction between a dead sensor and a real outage.
- Firing on scheduled load shedding.
- Ticket resolution based on someone clicking a button, with no telemetry verification.
- An LLM doing the fault localization. If you go this route, you had better have a strong argument, because a graph traversal is deterministic, instant, free, and explainable, and a language model is none of those.
- Claiming performance numbers you never measured.
- Documentation that describes a system other than the one in the repo.
- Building crew routing, auth, or analytics instead of the core.
- A repo you cannot explain.

After submission

Shortlisted candidates get a **30-minute call**. We will:

- Ask you to walk us through your localization algorithm.
- Pick two or three parts of your code and ask you to explain them — including parts an AI most likely wrote.
- Change the problem on you: another data source appears, or a constraint you relied on disappears. We want to see you think, not recite.
- Ask what you would do differently.

The call is about whether you understand what you shipped. Submissions built by someone who cannot explain them do not survive it, which is why we would rather see a smaller system you know completely than a large one you don't.

Questions and ground rules: [05-faq.md](#) ([05-faq.md](#)).